

CENTRO UNIVERSITÁRIO CATÓLICA DE SANTA CATARINA
CAMPUS JOINVILLE

CURSO DE ENGENHARIA DE SOFTWARE

MANUTENÇÃO E MELHORIA DE SOFTWARE
ANÁLISE DE REQUISITOS E MANUTENÇÃO
DO SISTEMA AGENDABEACH

Professora: Edicarsia Barbiero Pillon

JOINVILLE
2026

SUMÁRIO

1 INTRODUÇÃO	2
2 ESCOPO E REQUISITOS DO SISTEMA	3
2.1 Escopo	3
2.2 Regras de Negócio	3
2.3 Requisitos Funcionais	3
2.4 Requisitos Não Funcionais	4
3 ANÁLISE DO SISTEMA RECEBIDO	5
3.1 Estrutura Encontrada	5
3.2 Controle de Reservas	5
4 VERIFICAÇÃO DOS REQUISITOS	6
4.1 Regras de Negócio	6
4.2 Requisitos Funcionais	6
4.3 Requisitos Não Funcionais	7
5 MEDIDAS CORRETIVAS	8
5.1 Correção das Regras de Horário	8
5.2 Correção da Regra de Preços	8
5.3 Correção do Cancelamento	8
5.4 Implementação do Limite de Reservas	8
5.5 Tabela de Preços Administrável	8
5.6 Swagger/OpenAPI	8
6 MEDIDAS PREVENTIVAS	9
7 PROBLEMAS DE SEGURANÇA IDENTIFICADOS	10
8 PRIORIDADE DAS CORREÇÕES	11
9 CONCLUSÃO	12
10 REFERÊNCIAS	13

1 INTRODUÇÃO

Este documento apresenta a análise do sistema AgendaBeach, uma aplicação web destinada ao agendamento de quadras de Beach Tennis. A análise foi realizada considerando a especificação de requisitos fornecida para o projeto e o código-fonte disponível no repositório do sistema.

O objetivo foi verificar o que realmente está implementado, identificar diferenças entre os requisitos e o comportamento atual do sistema e registrar as medidas preventivas e corretivas necessárias para a manutenção do software.

A análise considera principalmente as regras de negócio RN01 a RN05, os requisitos funcionais RF01 a RF05 e os requisitos não funcionais RNF01 a RNF04 definidos na especificação.

2 ESCOPO E REQUISITOS DO SISTEMA

2.1 ESCOPO

O AgendaBeach consiste em uma aplicação web para agendamento de quadras de Beach Tennis. O sistema deve permitir que clientes visualizem a disponibilidade, realizem reservas e efetuem cancelamentos. Também deve permitir que administradores gerenciem o cadastro e o status das quadras.

2.2 REGRAS DE NEGÓCIO

Código	Regra
RN01	Quadras funcionando das 06:00 às 23:00, com agendamentos de duração fixa de 1 hora.
RN02	R\$ 60,00 por hora de segunda a sexta das 06:00 às 17:00 e R\$ 100,00 nos horários de pico, finais de semana e feriados.
RN03	Não permitir agendamentos duplicados para a mesma quadra e intervalo.
RN04	Mais de 24h: 100% de reembolso; entre 24h e 2h: 50% de crédito; menos de 2h: sem reembolso ou crédito.
RN05	Cada usuário pode possuir no máximo 3 agendamentos futuros em aberto.

2.3 REQUISITOS FUNCIONAIS

Código	Descrição
RF01	Exibir uma grade visual com horários disponíveis e ocupados por quadra e data.
RF02	Permitir que usuário autenticado selecione quadra, data e horário disponível e confirme o agendamento.
RF03	Permitir o cancelamento de reservas de acordo com as regras de antecedência.
RF04	Permitir ao administrador cadastrar, editar, inativar quadras e alterar a tabela de preços.
RF05	Permitir ao usuário consultar seus agendamentos passados e futuros com os respectivos status.

2.4 REQUISITOS NÃO FUNCIONAIS

Código	Descrição
RNF01	Consulta da grade de horários com tempo de resposta de até 2 segundos.
RNF02	Controle de concorrência para impedir que dois usuários reservem a mesma vaga simultaneamente.
RNF03	Interface responsiva para utilização em smartphones.
RNF04	Código organizado em camadas e documentação básica da API com Swagger/OpenAPI.

3 ANÁLISE DO SISTEMA RECEBIDO

3.1 ESTRUTURA ENCONTRADA

O projeto utiliza uma separação entre frontend, backend e banco de dados. O frontend foi desenvolvido com React, TypeScript, Vite e Mantine. O backend utiliza Java e Spring Boot, com Spring Web, Spring Data JPA e Spring Security. O banco de dados utilizado é o PostgreSQL.

O backend possui uma organização em Controller, Service, Repository, DTO, Entity e Enum. Essa estrutura está de acordo com a ideia de arquitetura em camadas definida na especificação.

O projeto também possui Docker, Docker Compose, GitHub Actions, ESLint, Prettier e SonarCloud. O README apresenta ainda regras de branch, commits, code review, Definition of Ready, Definition of Done e estratégia de testes.

3.2 CONTROLE DE RESERVAS

Uma parte importante da implementação está no banco de dados. A tabela de reservas utiliza uma coluna de intervalo temporal e uma restrição de exclusão para impedir sobreposição de reservas na mesma quadra quando o status é SCHEDULED. Essa estrutura atende diretamente ao objetivo da RN03 e também está alinhada com a modelagem indicada na especificação.

4 VERIFICAÇÃO DOS REQUISITOS

4.1 REGRAS DE NEGÓCIO

Req.	Situação	Análise
RN01	Não aplicado corretamente	O sistema utiliza início às 07:00 e permite durações de 1h, 1h30 e 2h, diferente da regra de 06:00–23:00 com duração fixa de 1h.
RN02	Não aplicado	O código utiliza valores e horários diferentes dos definidos na especificação. Foram encontrados valores de R\$ 100 e R\$ 120 e horário de pico diferente do requisito.
RN03	Aplicado	O PostgreSQL utiliza TSRANGE e EXCLUDE USING GIST para impedir sobreposição na mesma quadra.
RN04	Não aplicado corretamente	O sistema trabalha com faixas de 24h e 12h, enquanto o requisito define 24h e 2h. Também não foi identificada implementação do crédito de 50%.
RN05	Não aplicado	Não foi identificada estrutura de CPF nem validação do limite de três reservas futuras.

4.2 REQUISITOS FUNCIONAIS

Req.	Situação	Análise
RF01	Parcialmente aplicado	Existe calendário e consulta de reservas, porém a disponibilidade precisa ser ajustada para seguir integralmente os slots e horários da RN01.
RF02	Parcialmente aplicado	O fluxo de reserva existe, mas permite duração incompatível com RN01 e não contempla todos os controles de RN02 e RN05.
RF03	Parcialmente aplicado	Existe endpoint e tela de cancelamento, porém a regra aplicada é diferente da RN04.
RF04	Parcialmente aplicado	CRUD e status de quadras existem, mas a tabela de preços não é realmente administrável.
RF05	Aplicado	Existe consulta dos agendamentos do usuário e os status SCHEDULED, CANCELLED e COMPLETED.

4.3 REQUISITOS NÃO FUNCIONAIS

Req.	Situação	Análise
RNF01	Não comprovado	Existem índices no banco, mas não foi encontrada evidência de teste que comprove resposta inferior a 2 segundos.
RNF02	Parcialmente aplicado	A restrição do banco protege contra sobreposição, mas não foi identificado teste automatizado específico de concorrência.
RNF03	Aplicado	A interface foi construída com estrutura responsiva e componentes adequados para diferentes tamanhos de tela.
RNF04	Parcialmente aplicado	A arquitetura em camadas existe, porém não foi identificada implementação de Swagger/OpenAPI.

5 MEDIDAS CORRETIVAS

As medidas corretivas são necessárias para corrigir comportamentos que já estão diferentes dos requisitos definidos.

5.1 CORREÇÃO DAS REGRAS DE HORÁRIO

Alterar a geração dos horários para iniciar às 06:00 e encerrar às 23:00. A duração deve ser fixa em uma hora. A validação deve ocorrer principalmente no backend para impedir que uma chamada direta à API envie uma duração diferente.

5.2 CORREÇÃO DA REGRA DE PREÇOS

O cálculo deve ser ajustado para utilizar os valores definidos na RN02. A regra deve considerar dia da semana, horário e feriados. Os valores não devem ficar espalhados em diferentes partes do frontend e backend.

5.3 CORREÇÃO DO CANCELAMENTO

A regra deve ser alterada para 100% quando houver mais de 24 horas, 50% de crédito entre 24 e 2 horas e nenhum reembolso ou crédito abaixo de 2 horas. Também deve ser criada uma forma de registrar o crédito disponível para uso futuro.

5.4 IMPLEMENTAÇÃO DO LIMITE DE RESERVAS

É necessário incluir o CPF do usuário e verificar, antes de criar uma reserva, se ele já possui três agendamentos futuros em aberto. Essa validação deve ser realizada no backend.

5.5 TABELA DE PREÇOS ADMINISTRÁVEL

Os preços devem deixar de ser valores fixos no código e passar para uma estrutura configurável. Dessa forma, o administrador poderá alterar a tabela sem necessidade de modificar o código da aplicação.

5.6 SWAGGER/OPENAPI

Adicionar documentação da API para facilitar a manutenção e permitir que a equipe visualize e teste os endpoints disponíveis.

6 MEDIDAS PREVENTIVAS

As medidas preventivas têm como objetivo evitar que os mesmos problemas voltem a aparecer em futuras alterações do sistema.

- Transformar cada requisito em critério de aceite testável.
- Manter as regras de negócio críticas no backend.
- Criar testes unitários para preços, horários, cancelamentos e limite de reservas.
- Criar testes de integração para reserva e cancelamento.
- Criar teste concorrente para duas reservas no mesmo horário.
- Criar testes de desempenho para a consulta de disponibilidade.
- Manter code review obrigatório antes do merge.
- Executar automaticamente testes, lint, Prettier e análise de qualidade no CI.
- Atualizar a documentação da API sempre que um endpoint for alterado.
- Registrar débitos técnicos e pendências no backlog.

7 PROBLEMAS DE SEGURANÇA IDENTIFICADOS

Durante a análise foi identificado um ponto que deve ser tratado como prioridade. A aplicação utiliza o header X-User-Id para identificar o usuário e o código de segurança permite as requisições de forma ampla. Isso não representa uma autenticação e autorização adequada para um sistema que possui dados pessoais e funções administrativas.

Também existem endpoints de usuários que não apresentam uma proteção equivalente à utilizada nas operações administrativas. Isso pode permitir que uma requisição tente acessar dados de outro usuário apenas alterando um identificador.

Como correção, recomenda-se utilizar o Spring Security com autenticação real, como JWT ou sessão, e controlar as permissões por perfil. O usuário autenticado deve ser obtido pelo contexto de segurança e não por um ID enviado livremente pelo cliente.

Também foi identificado no frontend um mecanismo de detecção de perfil que utiliza uma operação administrativa para descobrir se o usuário é administrador. O ideal é que o perfil seja informado no processo de autenticação, evitando o uso de uma operação de negócio para descobrir permissão.

8 PRIORIDADE DAS CORREÇÕES

Prioridade	Correção	Motivo
Crítica	Autenticação e autorização	Reduzir risco de acesso indevido a dados e funções administrativas.
Crítica	RN01	Corrigir horário de funcionamento e duração das reservas.
Crítica	RN02	Corrigir valores e regras de cobrança.
Crítica	RN04	Corrigir política de cancelamento e crédito.
Alta	RN05	Implementar CPF e limite de três reservas futuras.
Alta	Crédito de cancelamento	Permitir o comportamento exigido pela RN04.
Alta	Preços configuráveis	Completar a gestão administrativa prevista no RF04.
Alta	Testes de concorrência	Comprovar o comportamento exigido pela RNF02.
Média	Swagger/OpenAPI	Melhorar a manutenção e documentação da API.
Média	Testes de frontend	Aumentar a cobertura automatizada.
Média	Teste de desempenho	Comprovar o limite de 2 segundos da RNF01.

9 CONCLUSÃO

A análise mostrou que o AgendaBeach possui uma estrutura de desenvolvimento já organizada, com frontend, backend, banco de dados, Docker, CI/CD e separação em camadas. Também existem funcionalidades principais implementadas, como cadastro, login, consulta de horários, reservas, cancelamento, histórico e gerenciamento de quadras.

Por outro lado, o sistema ainda apresenta diferenças importantes em relação aos requisitos definidos. As principais estão nas regras de horário, preço, cancelamento e limite de reservas. Também foram identificados pontos de segurança que precisam ser tratados antes de considerar o sistema pronto para uma utilização mais ampla.

A manutenção deve começar pelas correções críticas e depois avançar para os testes e melhorias de documentação. Dessa forma, além de corrigir os problemas atuais, o processo de desenvolvimento passa a reduzir a possibilidade de novos erros nas próximas alterações.

10 REFERÊNCIAS

Documento elaborado a partir da especificação de requisitos e da análise do código-fonte disponível no repositório do sistema AgendaBeach.